# ALIGARH MUSLIM UNIVERSITY

# COMPUTER LAB FILE

**Subject code: CABSMJ-2P02**

## Semester II

**Submitted to:**

- Dr. Mohammad Luqman

**Submitted by:**

Mohd Fahad Khan

Roll no.: 25CABSA573

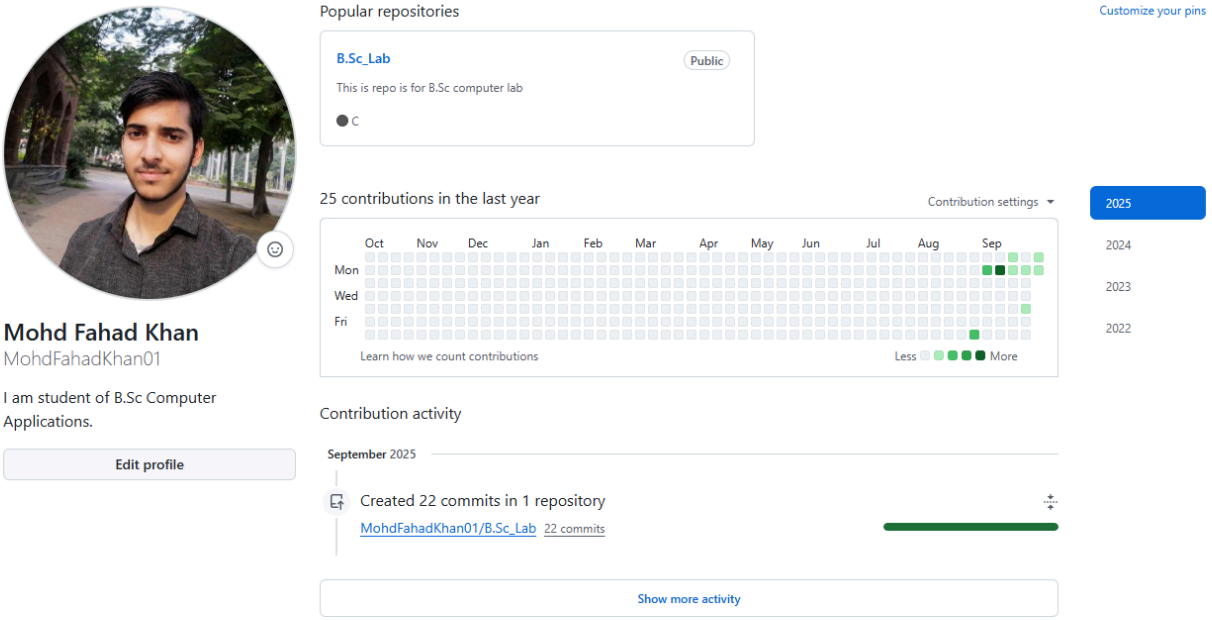
Enrol.no: GR0028

Department of Computer Science

Aligarh Muslim University

Session 2025-2026

# GitHub



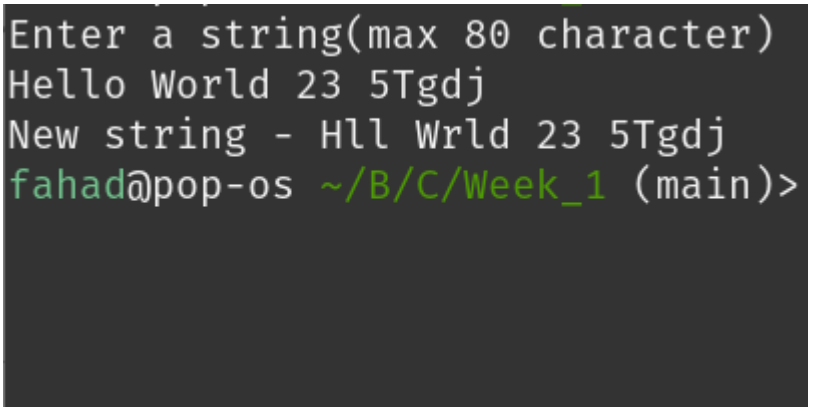

**Link- https://github.com/m-fahadk/B.Sc_Lab-2nd-Sem.git**

## SCAN

# Week- 1

## Q1- Write a C program to remove all vowels from a string.

```
#include <stdio.h>

int main()

{

   char str1[81], str2[81];

   printf("Enter a string(max 80 character)\n");

   fgets(str1, 81, stdin);

   int i = 0;

   int j = 0;

   char letter = str1[0];

   while (letter != '\0')

   {

      if (!(letter == 'A' || letter == 'E' || letter == 'I' || letter == 'O' || letter == 'U' || letter ==
'a' || letter == 'e' || letter == 'i' || letter == 'o' || letter == 'u'))

      {

         str2[j] = str1[i];

         i++;

         j++;

      }

      else

         i++;

      letter = str1[i];

   }

   str2[j] = '\0';

   printf("New string - ");

   printf("%s", str2);

}
```



## Q2- Write a C program to remove a given word from a string.

```
#include <stdio.h>
#include <string.h>

int main()
{
    char str1[81], str2[81], given_word[21], word2[21];
    str2[0] = '\0';

    printf("Enter a string(max 80 character)\n");
    fgets(str1, 81, stdin);

    printf("Enter a word to remove(max 20 character)\n");
    fgets(given_word, 21, stdin);
    given_word[strcspn(given_word, "\n")] = '\0';
    int i = 0;
    int j = 0;

    while (str1[i] != '\0' && str1[i] != '\n')
    {
        word2[j] = str1[i];
        i++, j++;

        if (str1[i] == ' ')
        {
            word2[j] = '\0';
```

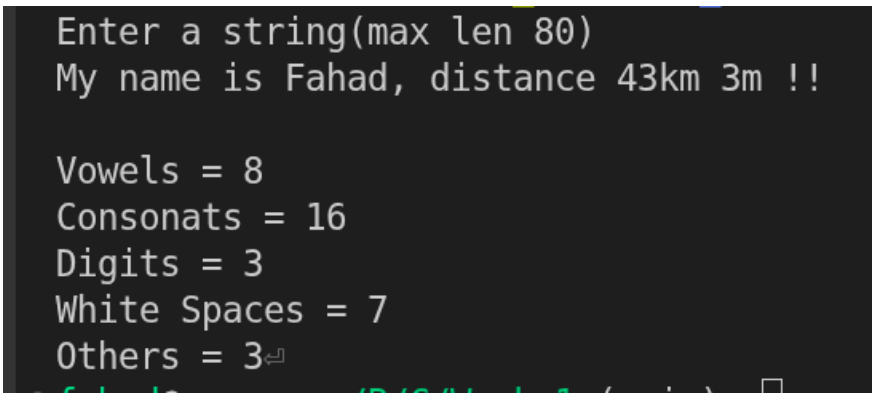
```
Enter a string(max 80 character)
This house is very big !
Enter a word to remove(max 20 character)
house
New sentence- This is very big !
fahad@pop-os ~/B/C/Week_1 (main)>
```

```
            if (!(strcmp(given_word, word2) == 0))
            {
               strcat(str2, word2);
               strcat(str2, " ");
            }
            j = 0;
            i++;
         }
      }
      word2[j] = '\0';
      if (!(strcmp(given_word, word2) == 0))
      {
         strcat(str2, word2);
      }
      printf("New sentence- %s", str2);
}
```

## Q3- Write a C program to find the number of vowels, consonants, digits, and white space character

```
#include <stdio.h>
#include <string.h>

int main()
{
   char str[81];
   int vowel_count = 0, consonant_count = 0, digit_count = 0,
white_space_count = 0, i = 0, others = 0;

   printf("Enter a string(max len 80)\n");
   fgets(str, 81, stdin);

   str[strcspn(str, "\n")] = '\0';
   while (str[i] != '\0')
   {
      if ((str[i] >= 'A' && str[i] <= 'Z') || (str[i] >= 'a' && str[i] <= 'z'))
      {
         if (str[i] == 'A' || str[i] == 'E' || str[i] == 'I' || str[i] == 'O' || str[i] == 'U' ||
str[i] == 'a' || str[i] == 'e' || str[i] == 'i' || str[i] == 'o' || str[i] == 'u')
            vowel_count++;
         else
            consonant_count++;
      }
      else if (str[i] >= '0' && str[i] <= '9')
```

```
            digit_count++;

        else if (str[i] == ' ' || str[i] == '\t')
            white_space_count++;

        else
            others++;

        i++;
    }
    printf("\nVowels = %d\n", vowel_count);
    printf("Consonats = %d\n", consonant_count);
    printf("Digits = %d\n", digit_count);
    printf("White Spaces = %d\n", white_space_count);
    printf("Others = %d", others);
}
```



Q4- Write a C program to perform the following operation in a matrix

a. Addition b. Subtraction

c. Multiplication d. Transpose

```
#include <stdio.h>
#include <string.h>
void input_matix(int m, int n, int arr[][n]){
printf("Enter the elements row wise :\n");
   for (int i = 0; i < m; i++) {
      for (int j = 0; j < n; j++){
         scanf("%d", *(arr + (i)) + j);
      } } }
void print_matrix(int m, int n, int (*arr)[n]){
   printf("\nMatrix\n");
   for (int i = 0; i < m; i++){
      for (int j = 0; j < n; j++){
         printf("%d ", *(*(arr + i) + j));
      }
      printf("\n");
   } }
int main(){
   printf("Choose a option: \n");
   printf("1 - Addition \n");
   printf("2 - Subtraction \n");
   printf("3 - Multiplication \n");
   printf("4 - Transpose \n");
   int choice, m, n, p;
```

```
Choose a option:
1 - Addition
2 - Subtraction
3 - Multiplication
4 - Transpose
1
Enter no. of rows and column of matrix :2 3
Enter the elements row wise :
1 2 3
6 4 9
Enter the elements row wise :
3 8 5
4 6 0

Matrix
4 10 8
10 10 9
```

```
scanf("%d", &choice);
switch (choice)  {
case 1:
{
   printf("Enter no. of rows and column of matrix :");
   scanf("%d%d", &m, &n);
   int arr1[m][n], arr2[m][n], res_arr[m][n];
   input_matix(m, n, arr1);
   input_matix(m, n, arr2);

   for (int i = 0; i < m; i++)
   {
      for (int j = 0; j < n; j++)
      {
         res_arr[i][j] = arr1[i][j] + arr2[i][j];
      }
   }
   print_matrix(m, n, res_arr);
   break;
}

case 2:
{
   printf("Enter no. of rows and column of matrix :");
   scanf("%d%d", &m, &n);
   int arr1[m][n], arr2[m][n], res_arr[m][n];
   input_matix(m, n, arr1);
   input_matix(m, n, arr2);
```

```
Choose a option:
1 - Addition
2 - Subtraction
3 - Multiplication
4 - Transpose
2
Enter no. of rows and column of matrix :3 2
Enter the elements row wise :
1 5
3 7
4 8
Enter the elements row wise :
0 5
5 4
3 2

Matrix
1 0
-2 3
1 6
```

```
    for (int i = 0; i < m; i++)
    {
        for (int j = 0; j < n; j++)
        {
            res_arr[i][j] = arr1[i][j] - arr2[i][j];
        }
    }
    print_matrix(m, n, res_arr);
    break;
}

case 3:
{
    printf("Enter no. of rows and column of ( 1st ) matrix :");
    scanf("%d%d", &m, &n);
    printf("2nd matix will have (%d) rows \n Enter the no. columns:  ", n);
    scanf("%d", &p);
    int arr1[m][n], arr2[n][p], res_arr[m][p];
    memset(res_arr, 0, sizeof(res_arr));

    input_matix(m, n, arr1);
    input_matix(n, p, arr2);

    for (int i = 0; i < m; i++)
    {
        for (int j = 0; j < p; j++)
        {
            for (int k = 0; k < n; k++)
            {
```

```
Choose a option:
1 - Addition
2 - Subtraction
3 - Multiplication
4 - Transpose
3
Enter no. of rows and column of ( 1st ) matrix :3 3
2nd matix will have (3) rows
 Enter the no. columns:  2
Enter the elements row wise :
1 2 3
4 5 6
7 8 9
Enter the elements row wise :
4 5
7 3
9 8

Matrix
45 35
105 83
165 131
```

```
                res_arr[i][j] += arr1[i][k] * arr2[k][j];
            }
        }
    }
    print_matrix(m, p, res_arr);
    break;
}

case 4:
{
    printf("Enter no. of rows and column of matrix :");
    scanf("%d%d", &m, &n);
    int arr1[m][n], res_arr[n][m];
    input_matix(m, n, arr1);

    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < m; j++)
        {
            res_arr[i][j] = arr1[j][i];
        }
    }
    print_matrix(n, m, res_arr);
    break;
}
default:
    printf("\nInvalid input");
}
}
```

```
Choose a option:
1 - Addition
2 - Subtraction
3 - Multiplication
4 - Transpose
4
Enter no. of rows and column of matrix :2 3
Enter the elements row wise :
1 5 8
5 4 0

Matrix
1 5
5 4
8 0
```

Q5- Write a C program to perform the following operation on strings using string functions.

a. Addition b. Copying

c. Reverse d. Length of String.

```
#include <stdio.h>
#include <string.h>
void strrev(char str[81]){
   char strcopy[81];
   strcpy(strcopy, str);
   int len = strlen(str);
   for (int i = 0; i < len; i++) {
      str[i] = strcopy[len - i - 1];
   }
}
int main(){
   char str1[81], str2[200];
   int choice;
   printf("Enter a string(max length 80) : ");
   fgets(str1, 81, stdin);
   char *postion = strchr(str1, '\n');
   if (postion != NULL)
   {
      *postion = '\0';
   }
   printf("Choose a option\n1- Addition\n2- Copying\n3- Reverse\n4- Length of string\n");
   scanf("%d", &choice);
```

```
Enter a string(max length 80) : This road is very long
Choose a option
1- Addition
2- Copying
3- Reverse
4- Length of string
1

Enter 2nd string(max length 80) :  wow !!

Result = This road is very long wow !!
```

```
    if (choice == 1)
    {
        getchar();
        printf("\nEnter 2nd string(max length 80) : ");
        fgets(str2, 200, stdin);
        strcat(str1, str2);
        printf("\nResult = %s", str1);
    }
    else if (choice == 2)
    {
        strcpy(str2, str1);
        printf("\nResult = %s", str2);
    }
    else if (choice == 3)
    {
        strrev(str1);
        printf("\nResult = %s", str1);
    }

    else if (choice == 4)
    {
        int l = strlen(str1);
        printf("Length of string = %ld", strlen(str1));
    }

    else
        printf("Invalid choice");
}
```

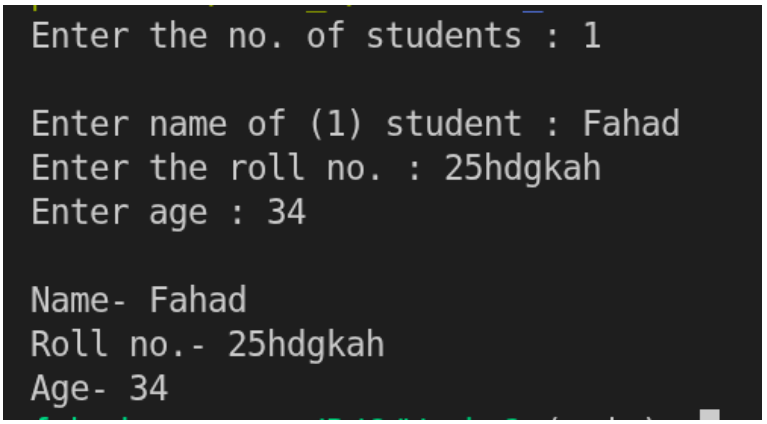
```
Enter a string(max length 80) : This road is very long
Choose a option
1- Addition
2- Copying
3- Reverse
4- Length of string
3

Result = gnol yrev si daor sihT
```

```
Enter a string(max length 80) : This road is very long
Choose a option
1- Addition
2- Copying
3- Reverse
4- Length of string
4
Length of string = 22
```

# Week- 2

Q1- Write a C program to store students' information on using a structure.

```
#include <stdio.h>


struct student_info
{
   char name[81];
   char roll_no[21];
   int age;
};
typedef struct student_info stud;


int main()
{
        int n,i;


        printf("Enter the no. of students : ");
        scanf("%d",&n);
        stud s[n];


        for(i=0;i<n;i++)
        {


                getchar();
           printf("\nEnter name of (%d) student : ",i+1);
           fgets(s[i].name, 81, stdin);
```



```
            printf("Enter the roll no. : ");
            fgets(s[i].roll_no, 21, stdin);

            printf("Enter age : ");
            scanf("%d", &s[i].age);

            printf("\nName- %sRoll no.- %sAge- %d\n", s[i].name, s[i].roll_no, s[i].age);
        }
}
```

## Q2- Write a C program to add two distances (in inch-feet) system using structure.

```
#include <stdio.h>
struct distances
{
   int feet;
   int inches;
};

int main()
{
   struct distances d1, d2, d3;

   printf("Enter the distance in (feet and inches) : ");
   scanf("%d%d", &d1.feet, &d1.inches);
   printf("Enter the 2nd distance in (feet and inches): ");
   scanf("%d%d", &d2.feet, &d2.inches);
   d3.feet = d1.feet + d2.feet;
   d3.inches = d1.inches + d2.inches;

   if (d3.inches >= 12)
   {
      d3.feet += d3.inches / 12;
      d3.inches = d3.inches % 12;
   }

   printf("Sum = %dFeet, %dinches", d3.feet, d3.inches);
}
```

```
Lab/Week_2/"distances
Enter the distance in (feet and inches) : 6 9
Enter the 2nd distance in (feet and inches): 3 6
Sum = 10Feet, 3inches
fahad@pop-os ~/B/C/Week_2 (main)>
```

## Q3- Write a C program to add two complex numbers by passing structure to a function.

```c
#include <stdio.h>
struct complex{
    float imaginary;
    float real;
};
typedef struct complex complex;

complex complex_sum(complex s1, complex s2){
    complex temp;
    temp.imaginary = s1.imaginary + s2.imaginary;
    temp.real = s1.real + s2.real;
    return temp;
}
int main(){
    complex s1, s2, result;
    printf("Enter the real and imaginary part : ");
    scanf("%f%f", &s1.real, &s1.imaginary);

    printf("Enter the real and imaginary part : ");
    scanf("%f%f", &s2.real, &s2.imaginary);
    result = complex_sum(s1, s2);
    printf("Result = %.2f + %.2fi", result.real, result.imaginary);
}
```

```
Week_2/"complex
Enter the real and imaginary part : 6 7
Enter the real and imaginary part : 8 2
Result = 14.00 + 9.00i
fahad@pop-os ~/B/C/Week_2 (main)>
```

# Q4- Write a C program to calculate the difference between the two time periods (HH: MM: SS).

```
#include <stdio.h>

struct time
{
   int hour;
   int min;
   int sec;
};
typedef struct time time;

time time_diff(time t1, time t2)
{
   time temp;
   t1.sec = t1.hour * 3600 + t1.min * 60 + t1.sec;
   t2.sec = t2.hour * 3600 + t2.min * 60 + t2.sec;
   if (t1.sec > t2.sec)
   {
      temp.sec = t1.sec - t2.sec;
   }

   else
   {
      temp.sec = t2.sec - t1.sec;
   }

   temp.hour = temp.sec / 3600;
   temp.sec = temp.sec % 3600;
```

```
    temp.min = temp.sec / 60;
    temp.sec = temp.sec % 60;

    return temp;
}

int main()
{
    time t1, t2, t3;

    printf("Enter the 1st time(HH/MM/SS) : ");
    scanf("%d%d%d", &t1.hour, &t1.min, &t1.sec);
    printf("Enter the 2nd time(HH/MM/SS) : ");
    scanf("%d%d%d", &t2.hour, &t2.min, &t2.sec);

    t3 = time_diff(t1, t2);
    printf("Time = %d : %d : %d", t3.hour, t3.min, t3.sec);
}
```

```
/"time
Enter the 1st time(HH/MM/SS) : 14 45 56
Enter the 2nd time(HH/MM/SS) : 12 34 54
Time = 2 : 11 : 2
fahad@pop-os ~/B/C/Week_2 (main)>
```

# Q5- Write a C program to store information using structures and pointers with dynamic memory allocation.

```
#include <stdio.h>
#include <stdlib.h>

struct commodity_data
{
   char name[81];
   int amount;
   float price;
};
typedef struct commodity_data co_data;

int main()
{
   int n;
   printf("Enter the no. of products : ");
   scanf("%d", &n);

   co_data *p = (co_data *)malloc(n * sizeof(co_data));

   for (int i = 0; i < n; i++)
   {
      getchar();
      printf("\nEnter the name of product: ");
      fgets((p + i)->name, sizeof((p + i)->name), stdin);

      printf("Enter the price(per kg): ");
```

```
Enter the no. of products : 2

Enter the name of product: Rice
Enter the price(per kg): 80
Enter the amount(in kg): 35

Enter the name of product: Wheat
Enter the price(per kg): 24
Enter the amount(in kg): 100

Name-  Rice
Price - 80.00
Amount- 35 kg
Name-  Wheat
Price - 24.00
```

```
        scanf("%f", &(p + i)->price);

        printf("Enter the amount(in kg): ");
        scanf("%d", &(p + i)->amount);
    }

    for (int i = 0; i < n; i++)
    {
        printf("\nName-  %s", (*(p + i)).name);

        printf("Price - %.2f", (*(p + i)).price);

        printf("\nAmount- %d kg", (*(p + i)).amount);
    }
}
```

# Week- 3

Q1- Write a C program that uses functions to perform the following operations on an array

a. Create an array of size 25 and insert the element of your choice

b. Delete the element at the beginning, middle, and end and adjust the Array.

c. Insert the element at the particular locations in the Array.

d. Display all the elements

```
#include <stdio.h>
void insertion(int a[], int n, int *m, int item, int k){
	int i;
	if (*m == (n - 1))
		printf("Array full (insertion not possible)\n");
	else if (k < 0 || k > (*m + 1))
		printf("Invalid value of index for insertion\n");
	else{
		for (i = *m; i >= k; i--)
			a[i + 1] = a[i];
		a[k] = item;
		*m = *m + 1;
	}}
void deletion(int a[], int *m, int k){
	int i;
	if (*m == -1)
```

```
MENU (Array Operations):
 1.Insertion at the specified index
 2.Deletion at the specified index
 3.Display the contents of array
 0.Exit
Choice :1
Enter an int value to be inserted: 35
Enter index no. ( 0 to 12): 5
```

```
            printf("Array empty (deletion not possible)\n");
        else if (k < 0 || k > (*m))
            printf("Invalid value of array index for deletion\n");
        else{
            for (i = k + 1; i <= (*m); i++)
                a[i - 1] = a[i];
            *m = *m - 1;
        }
}
void traversal(int a[], int n, int m){
        int i = 0;
        if (m == -1){
            printf("\nArray empty\n");
            return;
        }
        printf("\nContents of Array:\n");
        for (i = 0; i <= m; i++)
            printf("%d ", a[i]);
        printf("\n");
}
int main(){
        const int n = 25;
        int choice, item, mid, index;
        int arr[25] = {5, 7, 8, 9, 2, 5, 3, 6, 7, 2, 6, 2};
        int m = 11;

        do
        {
            printf("\nMENU (Array Operations):");
```

```
MENU (Array Operations):
 1.Insertion at the specified index
 2.Deletion at the specified index
 3.Display the contents of array
 0.Exit
Choice :3

Contents of Array:
5 7 8 9 2 35 5 3 6 7 2 6 2
```

```
            printf("\n 1.Insertion at the specified index");
            printf("\n 2.Deletion at the specified index");
            printf("\n 3.Display the contents of array");
            printf("\n 0.Exit");
            printf("\nChoice :");
            scanf("%d", &choice);

            switch (choice)
            {
            case 1:
                printf("Enter an int value to be inserted: ");
                scanf("%d", &item);
                printf("Enter index no. ( 0 to %d): ", m + 1);
                scanf("%d", &index);
                insertion(arr, n, &m, item, index);
                break;

            case 2:
                printf("Enter index no.: ( <= %d): ", m);
                scanf("%d", &index);
                deletion(arr, &m, index);
                break;

            case 3:
                traversal(arr, n, m);
                break;
            }
        } while (choice != 0);
}
```

```
MENU (Array Operations):
 1.Insertion at the specified index
 2.Deletion at the specified index
 3.Display the contents of array
 0.Exit
Choice :2
Enter index no.: ( <= 12): 5

MENU (Array Operations):
 1.Insertion at the specified index
 2.Deletion at the specified index
 3.Display the contents of array
 0.Exit
Choice :3

Contents of Array:
5 7 8 9 2 5 3 6 7 2 6 2
```

# Q2- Write a C Program to merge two sorted arrays into one sorted array.

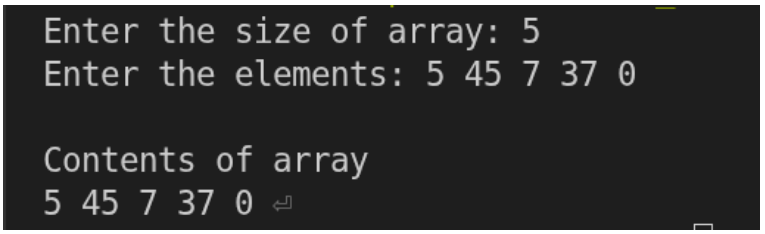
```
#include <stdio.h>
void array_print(int arr[], int n){
    int i;
    printf("\n\nContents of Array\n");
    for (i = 0; i < n; i++){
        printf("%d ", arr[i]);
    }
}
int main(){
    int i, j, l, n1, n2;
    printf("Enter no. of elements you want to enter for array 1: ");
    scanf("%d", &n1);
    int arr1[n1];
    printf("\nEnter the elements of arr1 in ascending order: ");
    for (i = 0; i < n1; i++){
        scanf("%d", &arr1[i]);
    }
    printf("\nEnter no. of elements you want to enter for array 2: ");
    scanf("%d", &n2);
    int arr2[n2];
    printf("\nEnter the elements of arr1 in ascending order: ");
    for (i = 0; i < n2; i++){
        scanf("%d", &arr2[i]);
    }
    int arr3[n1 + n2];
    i = 0, j = 0, l = 0;
```

```
Enter no. of elements you want to enter for array 1: 5

Enter the elements of arr1 in ascending order: 2 5 8 9 15

Enter no. of elements you want to enter for array 2: 4

Enter the elements of arr1 in ascending order: 4 7 67 353 565


Contents of Array
2 5 8 9 15

Contents of Array
4 7 67 353

Contents of Array
2 4 5 7 8 9 15 67 353
```

```
    while (i < n1 && j < n2){
        if (arr1[i] < arr2[j])
        {
            arr3[l] = arr1[i];
            i++;
        }
        else
        {
            arr3[l] = arr2[j];
            j++;
        }
        l++;
    }
    while (i < n1)
    {
        arr3[l] = arr1[i];
        i++, l++;
    }
    while (j < n2)
    {
        arr3[l] = arr2[j];
        j++;
        l++;
    }

    array_print(arr1, n1);
    array_print(arr2, n2);
    array_print(arr3, (n1 + n2));
}
```

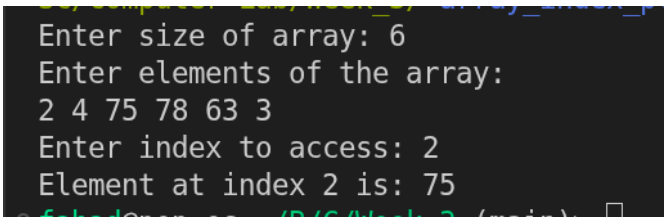
# Q3- Write a C Program to traverse the array using pointers.

```
#include <stdio.h>

int main()
{
   int n, i;
   printf("Enter the size of array: ");
   scanf("%d", &n);
   int arr[n];
   int *ptr = arr;

   printf("Enter the elements: ");
   for (i = 0; i < n; i++)
   {
      scanf("%d", ptr + i);
   }

   printf("\nContents of array\n");
   for (i = 0; i < n; i++)
   {
      printf("%d ", *(ptr + i));
   }
}
```



## Q4- Write a C Program that takes an index as input and prints the element of the array at that index using pointer arithmetic.

```
#include <stdio.h>
int main(){
    int n;
    printf("Enter size of array: ");
    scanf("%d", &n);
    int arr[n];
    int *ptr = arr;
    printf("Enter elements of the array:\n");
    for (int i = 0; i < n; i++)
        scanf("%d", &arr[i]);


    int index;
    printf("Enter index to access: ");
    scanf("%d", &index);


    if (index >= 0 && index < n)
    {
        printf("Element at index %d is: %d\n", index, *(ptr + index));
    }
    else
    {
        printf("Invalid index!\n");
    }


    return 0;
}
```



# Week- 4

Q1- Write a C program that uses functions to perform the following operations on a singly linked list

1# Creation

2# Insertion

- At beginning
- At middle
- At end

3# Deletion

- At beginning
- At middle
- At end

4# Traversal.

```
#include <stdio.h>
#include <stdlib.h>

struct node {
   int data;
   struct node *next;
};

struct node *head = NULL;

void insert_begin() {
   struct node *newnode;
   int value;
   newnode = (struct node *)malloc(sizeof(struct node));
   printf("Enter value: ");
   scanf("%d", &value);
   newnode->data = value;
   newnode->next = head;
```

```
    head = newnode;
    printf("Inserted at beginning.\n");
}

void insert_end() {
    struct node *newnode, *temp;
    int value;
    newnode = (struct node *)malloc(sizeof(struct node));
    printf("Enter value: ");
    scanf("%d", &value);
    newnode->data = value;
    newnode->next = NULL;
    if (head == NULL) {
        head = newnode;
        return;
    }
    temp = head;
    while (temp->next != NULL)
        temp = temp->next;
    temp->next = newnode;
    printf("Inserted at end.\n");
}

void insert_middle() {
    struct node *newnode, *temp;
    int value, pos, i;
    printf("Enter position: ");
    scanf("%d", &pos);
    if (pos == 1) {
        insert_begin();
        return;
    }
    newnode = (struct node *)malloc(sizeof(struct node));
    printf("Enter value: ");
    scanf("%d", &value);
    newnode->data = value;
    temp = head;
    for (i = 1; i < pos - 1 && temp != NULL; i++)
        temp = temp->next;
    if (temp == NULL) {
        printf("Invalid position\n");
```

```
===== LINKED LIST MENU =====
1. Insertion
2. Deletion
3. Traversal
4. Exit
Enter your choice: 1

Insertion at:
1. Beginning
2. Middle (position)
3. End
Enter option: 1
Enter value: 34
Inserted at beginning.

===== LINKED LIST MENU =====
1. Insertion
2. Deletion
3. Traversal
4. Exit
Enter your choice: 1

Insertion at:
1. Beginning
2. Middle (position)
3. End
Enter option: 3
Enter value: 0
Inserted at end.
```

```
        free(newnode);
        return;
    }
    newnode->next = temp->next;
    temp->next = newnode;
    printf("Inserted at position %d.\n", pos);
}

void delete_begin() {
    struct node *temp;
    if (head == NULL) {
        printf("List empty.\n");
        return;
    }
    temp = head;
    head = head->next;
    free(temp);
    printf("Deleted first node.\n");
}

void delete_end() {
    struct node *temp, *prev;
    if (head == NULL) {
        printf("List empty.\n");
        return;
    }
    if (head->next == NULL) {
        free(head);
        head = NULL;
        return;
    }
    temp = head;
    while (temp->next != NULL) {
        prev = temp;
        temp = temp->next;
    }
    prev->next = NULL;
    free(temp);
    printf("Deleted last node.\n");
}
```

```
===== LINKED LIST MENU =====
1. Insertion
2. Deletion
3. Traversal
4. Exit
Enter your choice: 1

Insertion at:
1. Beginning
2. Middle (position)
3. End
Enter option: 2
Enter position: 1
Enter value: 66
Inserted at beginning.

===== LINKED LIST MENU =====
1. Insertion
2. Deletion
3. Traversal
4. Exit
Enter your choice: 3
Linked List: 66 -> 34 -> 0 -> NULL
```

```
void delete_middle() {
    struct node *temp, *prev;
    int pos, i;
    printf("Enter position: ");
    scanf("%d", &pos);
    if (pos == 1) {
        delete_begin();
        return;
    }
    temp = head;
    for (i = 1; i < pos && temp != NULL; i++) {
        prev = temp;
        temp = temp->next;
    }
    if (temp == NULL) {
        printf("Invalid position\n");
        return;
    }
    prev->next = temp->next;
    free(temp);
    printf("Deleted node at position %d.\n", pos);
}

void traversal() {
    struct node *temp = head;
    if (head == NULL) {
        printf("List empty.\n");
        return;
    }
    printf("Linked List: ");
    while (temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

int main() {
    int choice, subchoice;
    do {
        printf("\n===== LINKED LIST MENU =====\n");
```

```
===== LINKED LIST MENU =====
1. Insertion
2. Deletion
3. Traversal
4. Exit
Enter your choice: 2

Deletion at:
1. Beginning
2. Middle (position)
3. End
Enter option: 1
Deleted first node.

===== LINKED LIST MENU =====
1. Insertion
2. Deletion
3. Traversal
4. Exit
Enter your choice: 2

Deletion at:
1. Beginning
2. Middle (position)
3. End
Enter option: 3
Deleted last node.
```

```
printf("1. Insertion\n");
printf("2. Deletion\n");
printf("3. Traversal\n");
printf("4. Exit\n");
printf("Enter your choice: ");
scanf("%d", &choice);

switch (choice) {
case 1:
   printf("\nInsertion at:\n");
   printf("1. Beginning\n");
   printf("2. Middle (position)\n");
   printf("3. End\n");
   printf("Enter option: ");
   scanf("%d", &subchoice);
   if (subchoice == 1)
      insert_begin();
   else if (subchoice == 2)
      insert_middle();
   else if (subchoice == 3)
      insert_end();
   else
      printf("Invalid insertion option.\n");
   break;

case 2:
   printf("\nDeletion at:\n");
   printf("1. Beginning\n");
   printf("2. Middle (position)\n");
   printf("3. End\n");
   printf("Enter option: ");
   scanf("%d", &subchoice);
   if (subchoice == 1)
      delete_begin();
   else if (subchoice == 2)
      delete_middle();
   else if (subchoice == 3)
      delete_end();
   else
      printf("Invalid deletion option.\n");
   break;
```

```
        case 3:
            traversal();
            break;

        case 4:
            printf("Exiting program...\n");
            break;

        default:
            printf("Invalid choice. Try again.\n");
        }
    } while (choice != 4);
    return 0;
}
```

# Week- 5

Q1- 1. Write a C program that uses functions to perform the following operations on a doubly linked list

1# Creation

2# Insertion

- At beginning
- At middle
- At end

3# Deletion

- At beginning
- At middle
- At end

4# Traversal.

```
#include <stdio.h>
#include <stdlib.h>

struct node {
   int data;
   struct node *prev;
   struct node *next;
};

struct node *head = NULL;

void insert_begin() {
   struct node *newnode;
   int value;
   newnode = (struct node *)malloc(sizeof(struct node));
   printf("Enter value: ");
   scanf("%d", &value);
   newnode->data = value;
```

```
    newnode->prev = NULL;
    newnode->next = head;
    if (head != NULL) head->prev = newnode;
    head = newnode;
    printf("Inserted at beginning.\n");
}

void insert_end() {
    struct node *newnode, *temp;
    int value;
    newnode = (struct node *)malloc(sizeof(struct node));
    printf("Enter value: ");
    scanf("%d", &value);
    newnode->data = value;
    newnode->next = NULL;
    if (head == NULL) {
        newnode->prev = NULL;
        head = newnode;
        return;
    }
    temp = head;
    while (temp->next != NULL) temp = temp->next;
    temp->next = newnode;
    newnode->prev = temp;
    printf("Inserted at end.\n");
}

void insert_middle() {
    struct node *newnode, *temp;
    int pos, i, value;
    printf("Enter position: ");
    scanf("%d", &pos);
    if (pos == 1) {
        insert_begin();
        return;
    }
    newnode = (struct node *)malloc(sizeof(struct node));
    printf("Enter value: ");
    scanf("%d", &value);
    newnode->data = value;
    temp = head;
```

```
===== DOUBLY LINKED LIST =====
1. Insertion
2. Deletion
3. Traversal
4. Exit
Enter choice: 1

1.Beginning
2.Middle
3.End
1
Enter value: 34
Inserted at beginning.

===== DOUBLY LINKED LIST =====
1. Insertion
2. Deletion
3. Traversal
4. Exit
Enter choice: 1

1.Beginning
2.Middle
3.End
3
Enter value: 64
Inserted at end.
```

```
    for (i = 1; i < pos - 1 && temp != NULL; i++) temp = temp->next;
    if (temp == NULL) {
        printf("Invalid position\n");
        free(newnode);
        return;
    }
    newnode->next = temp->next;
    newnode->prev = temp;
    if (temp->next != NULL) temp->next->prev = newnode;
    temp->next = newnode;
    printf("Inserted at position %d\n", pos);
}

void delete_begin() {
    struct node *temp;
    if (head == NULL) {
        printf("List empty.\n");
        return;
    }
    temp = head;
    head = head->next;
    if (head != NULL) head->prev = NULL;
    free(temp);
    printf("First node deleted.\n");
}

void delete_end() {
    struct node *temp;
    if (head == NULL) {
        printf("List empty.\n");
        return;
    }
    temp = head;
    while (temp->next != NULL) temp = temp->next;
    if (temp->prev != NULL) temp->prev->next = NULL;
    else head = NULL;
    free(temp);
    printf("Last node deleted.\n");
}

void delete_middle() {
```

```
===== DOUBLY LINKED LIST =====
1. Insertion
2. Deletion
3. Traversal
4. Exit
Enter choice: 1

1.Beginning
2.Middle
3.End
2
Enter position: 2
Enter value: 0
Inserted at position 2

===== DOUBLY LINKED LIST =====
1. Insertion
2. Deletion
3. Traversal
4. Exit
Enter choice: 3
List: 34 <-> 0 <-> 64 <-> NULL
```

```
    struct node *temp;
    int pos, i;
    printf("Enter position: ");
    scanf("%d", &pos);
    if (pos == 1) {
        delete_begin();
        return;
    }
    temp = head;
    for (i = 1; i < pos && temp != NULL; i++) temp = temp->next;
    if (temp == NULL) {
        printf("Invalid position\n");
        return;
    }
    if (temp->prev != NULL) temp->prev->next = temp->next;
    if (temp->next != NULL) temp->next->prev = temp->prev;
    free(temp);
    printf("Node deleted at position %d\n", pos);
}

void traversal() {
    struct node *temp = head;
    if (head == NULL) {
        printf("List empty.\n");
        return;
    }
    printf("List: ");
    while (temp != NULL) {
        printf("%d <-> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

int main() {
    int choice, subchoice;
    do {
        printf("\n===== DOUBLY LINKED LIST =====\n");
        printf("1. Insertion\n2. Deletion\n3. Traversal\n4. Exit\nEnter choice: ");
        scanf("%d", &choice);
```

```
===== DOUBLY LINKED LIST =====
1. Insertion
2. Deletion
3. Traversal
4. Exit
Enter choice: 2

1.Beginning
2.Middle
3.End
1
First node deleted.

===== DOUBLY LINKED LIST =====
1. Insertion
2. Deletion
3. Traversal
4. Exit
Enter choice: 2

1.Beginning
2.Middle
3.End
3
Last node deleted.
```

```
        switch (choice) {
        case 1:
            printf("\n1.Beginning\n2.Middle\n3.End\n");
            scanf("%d", &subchoice);
            if (subchoice == 1) insert_begin();
            else if (subchoice == 2) insert_middle();
            else if (subchoice == 3) insert_end();
            else printf("Invalid option\n");
            break;

        case 2:
            printf("\n1.Beginning\n2.Middle\n3.End\n");
            scanf("%d", &subchoice);
            if (subchoice == 1) delete_begin();
            else if (subchoice == 2) delete_middle();
            else if (subchoice == 3) delete_end();
            else printf("Invalid option\n");
            break;

        case 3:
            traversal();
            break;

        case 4:
            printf("Program ended.\n");
            break;

        default:
            printf("Invalid choice\n");
        }
    } while (choice != 4);
    return 0;
}
```